Assignment 2

A) Changing the Number of Hidden Layers (1, 2, 3, 4, and 5)

The original code provided in the assignment for building the neural network model requires some modifications. First, we need to set the input_shape parameter in the first layer. Then, we need to set the number of units in the output layer to 1. Since the goal is to detect whether a message is spam or not, the output layer must have a single unit to perform Binary Classification correctly.

The modified code is shown below:

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(10, activation="relu",
input_shape=(vocab_size,)),
    layers.Dense(5, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
```

This model consists of three layers:

- Input layer: With 10 units.
- Hidden layer: With 5 units and the ReLU activation function.
- Output layer: With 1 unit and the Sigmoid activation function.

The Sigmoid activation function produces an output value between 0 and 1, indicating whether a message is spam or not.

We then train the model using the specified number of epochs and batch_size provided in the assignment.

```
model.fit(x_train, y_train, epochs=4, batch_size=512)
```

We observe that, in this configuration, the Test Accuracy is 0.889479.

Next, we increase the number of hidden layers and present the model's accuracy for different numbers of hidden layers in the table below. Finally, we analyze and compare the results.

The additional hidden layers in each experiment have exactly the same specifications as the first hidden layer. In other words, for each additional layer, we add another line of code representing a hidden layer with 5 units and the ReLU activation function.

Accuracy	Number of hidden layers
0.882399	1
0.888199	2
0.883599	3
0.887639	4
0.874000	5

We observe that increasing the number of hidden layers does not lead to a significant change in the model's accuracy. This may be due to overfitting.

Overfitting: Adding more hidden layers increases the model's capacity to learn complex patterns from the training data. However, if the model becomes too complex relative to the size of the training dataset (in this case, by increasing the number of hidden layers), it may fit the training data too closely and focus excessively on specific details. Ultimately, this can lead to a decrease in the model's accuracy on unseen data.

Therefore, we conclude that increasing the number of hidden layers does not necessarily improve the model's accuracy. In this case, the model with two hidden layers achieves better accuracy than the other configurations.

B) Changing the Size of the Hidden Layers (at least three different values in the range of 10 to 1024)

The size of a hidden layer refers to the number of neurons, or units, in that layer. To change the size of the hidden layers, we use the code below. The hidden layers are indicated by comments, and the numerical value specified in the layers.Dense argument represents the number of neurons in each hidden layer.

We vary this value within the range of 10 to 1024, evaluate the accuracy of each model, and record the results in the table below.

```
model = keras.Sequential([
    layers.Dense(10, activation="relu", input_shape=(vocab_size,)),
    layers.Dense(5, activation="relu"),  #First Hidden Layer
    layers.Dense(5, activation="relu"),  #Second Hidden Layer
```

```
    layers.Dense(1, activation="sigmoid")
])
```

accuracy	Size of hidden layers
0.8660799	10
0.8701199	128
0.8777999	256
0.8778799	512
0.8785600	1024

By using model.summary(), we can observe that as the size of the hidden layers increases, the number of parameters also increases. Furthermore, the accuracy improves across successive epochs (for example, the accuracy in epochs 2, 3, and 4 is higher than that in epoch 1).

We also observe that within the range of 10 to 1024, increasing the size of the hidden layers leads to a slight improvement in the model's accuracy. However, an important point to consider is that increasing the size of the hidden layers also increases the number of parameters and, consequently, the complexity of the model. This may eventually lead to overfitting.

Another factor I considered was the model's training time when executing the fit() function in the following line of code. With a hidden layer size of 128, the code took 6 seconds to run, whereas with a size of 256, it took 8 seconds. Since the number of epochs is relatively small in this experiment, the difference may not seem significant. However, as the number of epochs increases, the difference in execution time between these two configurations will likely become more noticeable.

```
model.fit(x_train, y_train, epochs=4, batch_size=512)
```

Considering that the accuracy in these two configurations differs only slightly, in the third decimal place, this suggests that increasing the size of the hidden layers does not necessarily result in a significant improvement in accuracy. In some cases, it may only lead to minor improvements while increasing execution time and neural network complexity. Therefore, the additional computational cost may not always be worthwhile.

C) Changing the Vocabulary Size

Changing the vocabulary size (vocab_size) means changing the number of words considered by the model in the IMDB dataset. A smaller value means that the model supports fewer words.
The table below shows how changing the vocabulary size affects the model's accuracy. Initially, the vocabulary size is set to 10,000.
Using the following code, we reduce the vocabulary size by 20%, 50%, and 80% to evaluate and compare the results.

```
from tensorflow.keras.datasets import imdb

original_vocab_size = 10000

#Reduce vocabulary size by 80%
vocab_size_80_percent_reduction = int(original_vocab_size * 0.2)
(train_data_80, train_labels_80), (test_data_80, test_labels_80) =
imdb.load_data(num_words=vocab_size_80_percent_reduction)
#Reduce vocabulary size by 50%
vocab_size_50_percent_reduction = int(original_vocab_size * 0.5)
(train_data_50, train_labels_50), (test_data_50, test_labels_50) =
imdb.load_data(num_words=vocab_size_50_percent_reduction)
#Reduce vocabulary size by 20%
vocab_size_20_percent_reduction = int(original_vocab_size * 0.8)
(train_data_20, train_labels_20), (test_data_20, test_labels_20) =
imdb.load_data(num_words=vocab_size_20_percent_reduction)
```

```
from tensorflow import keras
from keras import layers
from keras import activations

model = keras.Sequential([
    layers.Dense(10, activation="relu", input_shape=(vocab_size,)),
    layers.Dense(256, activation="relu"),  #First Hidden Layer
    layers.Dense(256, activation="relu"),  #Second Hidden Layer
    layers.Dense(256, activation="relu"),  #Third Hidden Layer
    layers.Dense(1, activation="sigmoid")
])
```

Accuracy	Reduction in vocabulary size(in percentage)
0.8727999	vocab size = 8000 - %2۰
0.8657600	vocab size = 5000 - %۵۰
0.8511599	vocab size = 2000 - %۸۰

Now we increase the vocabulary size:

Accuracy	Increase in vocabulary size(in percentage)
0.8501600	vocab size = 12000 - %2۰
0.8745200	vocab size = 15000 - %۵۰
0.8794000	vocab size = 18000 - %۸۰

We observe that as the vocabulary size decreases, the model's accuracy also declines. This is mainly due to the reduction in the number of known words and the smaller number of keywords available to the model. As a result, the model has less information to understand the text and its context, which ultimately reduces its accuracy in classifying messages as spam or not spam.
Conversely, increasing the vocabulary size allows the model to recognize a larger number of words in the training documents (i.e., known words), improving its ability to understand the document contents. As a result, the model's accuracy increases. However, increasing the vocabulary size does not necessarily always lead to better accuracy, as this depends on various factors, including the type of problem.

D) Changing the Activation Functions of the Hidden Layers (Sigmoid, Linear, and ReLU)

In the original code, the activation function used in the hidden layers is ReLU. ReLU works by returning the input value for positive values greater than zero and returning zero for negative values. It is defined as follows:

$$F(x) = x \text{ if } x>0 \text{ , } F(x) = 0 \text{ if } x<0$$

The Sigmoid function returns a value between 0 and 1, whereas the Linear function is an identity function, meaning that it returns the input value unchanged.

In the following code, we change the activation functions used in the hidden layers.

```
from tensorflow import keras
from keras import layers
from keras import activations

model = keras.Sequential([
```

```
    layers.Dense(10, activation="relu", input_shape=(vocab_size,)),
    layers.Dense(256, activation="relu"),  #First Hidden Layer
    layers.Dense(256, activation="relu"),  #Second Hidden Layer
    layers.Dense(256, activation="relu"),  #Third Hidden Layer
    layers.Dense(1, activation="sigmoid")
])
```

The table below shows how the model's accuracy changes with different activation functions in the hidden layers.

دقت مدل	Activation Function
0.8769199	Relu
0.7658399	Sigmoid
0.8728799	Linear

According to the results, the Sigmoid function is suitable for the output layer in binary classification problems, but it does not perform particularly well in hidden layers and noticeably reduces the model's accuracy.
The Linear function returns each input value unchanged. It is commonly used in the output layer of regression problems. Although it also performs reasonably well in the hidden layers in this experiment, using ReLU as the activation function for the hidden layers achieves the highest accuracy among the tested options. Therefore, ReLU proves to be the most suitable choice for the hidden layers of this neural network.

Based on the results obtained, we proceed as shown in the table below. Due to Google Colab's limitations and memory errors, we kept the vocabulary size at the same value used in the original code and decided not to use a vocabulary size of 18,000, since the resulting improvement in accuracy was only marginal.

2	Number of hidden layers
1024	Size of hidden layers
10000	Size of vocabulary
relu	Activation Function in Hidden Layers

The model achieves an accuracy of 0.8766000.

We could also have increased the batch_size and the number of epochs. However, when we increased the number of epochs to 10, we observed signs of overfitting. Although the accuracy improved with each epoch, the model's final performance on the test data decreased.

Using the same code, I also increased the number of hidden layers to investigate whether this would improve the model's performance. However, the accuracy did not improve; instead, it decreased. I also tested different hidden layer sizes, and a size of 1024 produced the best results. Additionally, I tested batch_size values of 256 and 128. Nevertheless, the model trained with the configuration provided in the nlp.ipynb file still achieved better accuracy.

One important point to consider is that higher accuracy during training does not necessarily mean that the model will achieve higher accuracy on the test data.
For example, the output below shows the accuracy at each epoch when using a batch_size of 256. We can observe that the accuracy at each epoch is higher than the accuracy reported in the corresponding section of the nlp.ipynb file. However, the final test accuracy of the model trained with a batch_size of 256 is lower than that of the model in the provided file.
This may indicate overfitting. With a batch_size of 256, the model may have learned the training data too closely, achieving higher accuracy on those same training samples. However, when presented with new, unseen data, it may fail to analyze it effectively, resulting in lower classification accuracy.

```
model.fit(x_train, y_train, epochs=4, batch_size=256)
Epoch 1/4
98/98 [==============================] - 7s 67ms/step - loss: 0.0222 -
accuracy: 0.9923
Epoch 2/4
98/98 [==============================] - 5s 56ms/step - loss: 0.0225 -
accuracy: 0.9931
Epoch 3/4
98/98 [==============================] - 7s 67ms/step - loss: 0.0140 -
accuracy: 0.9951
Epoch 4/4
 98/98 [==============================] - 6s 56ms/step - loss: 0.0204
                                                    - accuracy: 0.9931
```

```
test_acc: 0.8616399765014648      test_loss: 0.8915560841560364
```